

EMATM0048: Software Development Programming and Algorithms (SDPA)

Tutorial 1

Please use either Spyder or Jupyter Notebook to implement the code below

In this tutorial we will:

- Explore the choices available to run Python apps
- Learn how to take input and produces output
- Convert between types
- Explore the use of basic operators

1. Variables and basic data types in Python

In Python, a variable is declared and assigned a value using the assignment operator `=`. The variable being assigned to, is on the left-hand side of the operator, and the value being assigned, which can be an expression such as `2 + 2` and can even include other variables, is on the right-hand side:

```
In [ ]: x = 1          # assign variable x the value 1
        y = x + 5      # assign variable y the value of x plus 5
        z = y          # assign variable z the value of y
```

```
In [ ]: print (x,y,z)
```

Assignments can be done on more than one variable "simultaneously" on the same line like this

```
In [ ]: a, b = 3, 4
        print(a,b)
```

Working with numbers

Most programs manipulate numbers. Computers treat integer numbers and decimal numbers differently. Consider the following code:

```
In [ ]: x = 1          # integer
        x = 1.0        # decimal (floating point)
```

Python creates integers from a built-in data type called `int`, and decimals (floating-point numbers) as instances of `float`. Python's built-in `type()` function returns a variable's data type. The following code outputs types to the screen:

```
In [ ]: x = 1
        print(type(x)) # outputs: <class 'int'>
```

```
x = 1.0
print(type(x)) # outputs: <class 'float'>
```

The addition of the ".0" to the end of "1" makes a large difference in how the programming language treats a value. The data type impacts how the value is stored in memory, how the processor (CPU) handles the data when evaluating expressions, how the data relates to other data, and what kinds of operations can be performed with it.

Another common data type is the Boolean type, which holds the value True or False:

```
In [ ]: x = True
        print(type(x)) # outputs: <class 'bool'>
```

Internally, bool is treated as a special type of integer. Technically, True has a value of 1 and False has a value of 0. Typically, Booleans are not used to perform mathematical operations; rather, they are used to make decisions and perform branching. Nevertheless, it is interesting to understand the relationship between types. Many types are nothing more than specialized versions of more general types. Integers are a subset of floating point numbers, and Booleans are a subset of integers.

2. Reading keyboard input

Many programs are interactive. Supporting interactivity means you have a program that runs differently depending on what the input is. The one inputting data to a program is usually a user, but it can be another program for example. There are many ways to send input to a program, two common ways are via a graphical interface or a console.

For reading input from the keyboard, Python provides the `input()` function. `input()` reads what the user types on the keyboard and returns it as a string. Here is an example that combines `input()` and `print()` to capture a person's name and then display it on the screen:

```
In [ ]: name = input('Enter your name: ')
        print(name)
```

The string passed as an argument to the `input` function is the prompt that the user will see. In this example, you are asking the user to type their name ('Enter your name'). Once the user types a name and presses Enter, the `input` function will return. The function's return value is the text that the user typed, and that text is assigned to the variable named `name`. The `name` variable is then used as an input or argument to the `print` function, which will output the name that the user entered.

You can also call the `input` function without a parameter:

```
In [ ]: print('What is your name?')
        name = input()
        print(name)
```

This program will behave almost the same as the first one. The difference is that `print` (by default) adds a newline to the output.

Reading a number

The `input` function always returns the typed value as a string (text). This choice makes sense because the user can enter whatever value they like. Even if the input is a valid number, it's still returned as a string type from the input function. For example:

```
In [ ]: x = input('Enter a number: ')
        print(type(x))
```

Running this code and entering the value '5' would display `<class 'str'>` even though the value itself is numeric. To turn the value into a true integer variable, you can use the `int()` function:

```
In [ ]: x = int(input('Enter a number: '))
        print(type(x))
```

Note: What if the input isn't numeric and you pass it to the `int()` function? As you might expect, this would be an error and cause a runtime failure. The program will end at this statement - you can try it yourself in the Python interactive console. We'll cover various solutions to handling these sorts of errors in future modules.

To see how to use the print command, you may find it convenient to look at the input and output section of the Python Wikibook

https://en.wikibooks.org/wiki/Python_Programming/Input_and_Output. This will show you how to use print statements to print out values of strings.

3. Convert types

Converting numbers to strings

You can go the other direction as well. The `str()` method will take an integer or float value and turn it into a string. Calling the `str()` method is needed if you want the below code example to work. The conversion ensures the integer, in its string form, is concatenated to the string on the left.

```
In [ ]: x = 5
        print('The number is ' + str(x))
```

Simple operators can be executed on numbers and strings:

```
In [ ]: one = 1
        two = 2
        three = one + two
        print(three)

        hello = "hello"
        world = "world"
        helloworld = hello + " " + world
        print(helloworld)
```

Mixing operators between numbers and strings is not supported:

```
In [ ]: # This will not work!
        one = 1
        two = 2
        hello = "hello"

        print(one + two + hello)
```

4. Operators

For a full list of operators <https://docs.python.org/3/library/operator.html>

- Arithmetic operators

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 30$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = -10$
* Multiplication	Multiplies values on either side of the operator	$a * b = 200$
/ Division	Divides left hand operand by right hand operand	$b / a = 2$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 0$
** Exponent	Performs exponential (power) calculation on operators	$a ** b = 10 \text{ to the power } 20$
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity)	$9 // 2 = 4$ and $9.0 // 2.0 = 4.0$, $-11 // 3 = -4$, $-11.0 // 3 = -4.0$

- Python Comparison Operators

Operator	Description	Example
<code>==</code>	If the values of two operands are equal, then the condition becomes true.	<code>(a == b)</code> is not true.
<code>!=</code>	If values of two operands are not equal, then condition becomes true.	<code>(a != b)</code> is true.
<code><></code>	If values of two operands are not equal, then condition becomes true.	<code>(a <> b)</code> is true. This is similar to <code>!=</code> operator.
<code>></code>	If the value of left operand is greater than the value of right operand, then condition becomes true.	<code>(a > b)</code> is not true.
<code><</code>	If the value of left operand is less than the value of right operand, then condition becomes true.	<code>(a < b)</code> is true.
<code>>=</code>	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.	<code>(a >= b)</code> is not true.
<code><=</code>	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.	<code>(a <= b)</code> is true.

```
In [ ]: number = 1 + 2 * 3 / 4.0
        print(number)
```

Another operator available is the modulo (%) operator, which returns the integer remainder of the division. `dividend % divisor = remainder`.

```
In [ ]: remainder = 11 % 3
        print(remainder)
```

Watch the output of the following:

```
In [ ]: 17 / 3 # classic division returns a float
```

```
In [ ]: 17 // 3 # floor division discards the fractional part
```

```
In [ ]: 17 % 3 # the % operator returns the remainder of the division
```

Exercise: Using remainder and floor division, how to get the original number?

```
In [ ]:
```

Exercises:

- Start the Python interpreter and use it as a calculator.
- . Write a program to take a number and return the squared and cubic of it

```
In [ ]: #Sol1:
```

. Write a Python program to calculate body mass index

```
In [ ]: #Sol 2
```

. Write a Python program to solve $(x + y) * (x + y)$.

- ♦ example : $x = 4, y = 3$ Expected Output : $(4 + 3) ^ 2 = 49$

```
In [ ]: #Sol 3
```

. What time it would be nine hours after 8 00 a.m. On a twelve-hour clock? Write a python code to answer that

```
In [ ]: #Sol 4
```

- Input and output exercise:

. Store a person's name in a variable, and print a message to that person. Your message should be simple, such as, "Hello Eric, would you like to learn some Python today?"

```
In [ ]: #Sol 1
```

. Write a Python program which accepts the user's first and last name and print them in reverse order with a space between them

```
In [ ]: #2
```

- Challenge yourself:

. Write a Python program that accepts an integer (n) and computes the value of $n+nn+nnn$. Sample value of n is 5 Expected Result : 615

```
In [ ]:
```

. Write a Python program to find whether a given number (accept from the user) is even or odd, print out an appropriate message to the user.

```
In [ ]:
```

. Write a Python program to calculate the sum of the digits in an integer (of 4 digits).

```
In [ ]:
```

. If I leave my house at 6 52 am and run 1 mile at an easy pace (8 15 per mile), then 3 miles at tempo (7 12 per mile) and 1 mile at easy pace again, what time do I get home for breakfast?

```
In [ ]:
```

More exercises and references

- https://www.learnpython.org/en/Variables_and_Types
- <https://pynative.com/python/> -
https://en.wikibooks.org/wiki/Python_Programming/Input_and_Output -
https://en.wikibooks.org/wiki/Python_Programming/Variables_and_Strings
- https://en.wikibooks.org/wiki/Python_Programming/Basic_Math